Unidade didáctica 3. Consultas SQL (II)

Nesta parte da unidade continuaremos traballando coas consultas SQL. Aprenderemos a limitar os resultados, utilizar alias e expresións, realizar operacións de resumo e agrupar rexistros.

Tamén estudaremos como recuperar información almacenada en varias táboas mediante composicións internas (`INNER JOIN`), utilizando as relacións establecidas entre as claves primarias e foráneas.

7. Limitación de resultados, alias e expresións

7.1. Limitación de resultados con LIMIT

Cando realizamos unha consulta, normalmente obtense todos os rexistros que cumpren as condicións establecidas. Non obstante, en determinadas situacións pode interesarnos recuperar unicamente unha parte dos resultados.

A cláusula `LIMIT` permite establecer o número máximo de rexistros que devolverá unha consulta.

```
SELECT columnas
FROM nome_taboa
LIMIT numero;
```

Exemplo 1. Mostrar os tres primeiros alumnos

```
SELECT *
FROM alumnos
LIMIT 3;
```

A consulta devolverá como máximo tres rexistros da táboa `alumnos` .

> **Importante:** Sen utilizar `ORDER BY` , non debemos asumir que os rexistros devoltos serán sempre os mesmos nin que seguirán unha orde determinada.

7.2. LIMIT combinado con ORDER BY

É habitual combinar `LIMIT` con `ORDER BY` para recuperar os primeiros rexistros segundo un criterio de ordenación.

Exemplo 2. Os tres alumnos de maior idade

```
SELECT *
FROM alumnos
ORDER BY idade DESC
LIMIT 3;
```

Primeiro ordénanse os alumnos de maior a menor idade e posteriormente recupéranse os tres primeiros.

Exemplo 3. Os dous cursos con maior número de horas

```
SELECT *
FROM cursos
ORDER BY horas DESC
LIMIT 2;
```

7.3. LIMIT con desprazamento (OFFSET)

Podemos indicar non só cantos rexistros queremos recuperar, senón tamén cantos rexistros queremos saltar antes de comezar a mostrar os resultados.

```
SELECT columnas
FROM nome_taboa
LIMIT numero OFFSET desprazamento;
```

O valor de `OFFSET` indica cantos rexistros se deben saltar antes de comezar a devolver os resultados.

Exemplo 4. Mostrar tres alumnos saltando os dous primeiros

```
SELECT *
FROM alumnos
ORDER BY id
LIMIT 3 OFFSET 2;
```

A consulta ordena os alumnos polo identificador, salta os dous primeiros e devolve os tres seguintes.

Tamén podemos utilizar a sintaxe abreviada:

```
SELECT *
FROM alumnos
ORDER BY id
LIMIT 2, 3;
```

Nesta sintaxe, o primeiro número representa o desprazamento e o segundo o número máximo de rexistros que se devolverán.

> **Para lembrar:**
>
> - `LIMIT 5` : devolve como máximo cinco rexistros.
> - `LIMIT 5 OFFSET 10` : salta dez rexistros e devolve cinco.
> - `LIMIT 10, 5` : equivalente ao caso anterior.

7.4. Alias de columnas

Un alias é un nome alternativo que podemos asignar a unha columna no resultado dunha consulta.

Utilízase para facer que os resultados sexan máis claros e comprensibles, especialmente cando traballamos con expresións ou funcións de resumo.

```
SELECT columna AS nome_alias
FROM nome_taboa;
```

Exemplo 1. Cambiar o nome dunha columna

```
SELECT nome AS NomeAlumno
FROM alumnos;
```

A columna aparecerá no resultado co nome `NomeAlumno`, pero o nome real do campo na táboa continuará sendo `nome`.

A palabra reservada `AS` é opcional.

```
SELECT nome NomeAlumno
FROM alumnos;
```

Se o alias contén espazos, debemos utilizar comiñas invertidas (acento grave).

```
SELECT nome AS `Nome do alumno`
FROM alumnos;
```

> **Importante:** Os alias modifican o nome co que se presenta unha columna no resultado da consulta. Non modifican a estrutura da táboa nin cambian o nome real do campo.

7.5. Alias de táboas

Tamén podemos asignar nomes alternativos ás táboas durante a execución dunha consulta.

```
SELECT a.nome
FROM alumnos AS a;
```

Neste exemplo, `a` é un alias da táboa `alumnos`. Podemos utilizalo para referirnos ás súas columnas.

Os alias de táboas son especialmente útiles cando traballamos con varias táboas ou cando existen columnas co mesmo nome en diferentes táboas.

7.6. Expresións nas consultas

SQL permite realizar operacións matemáticas directamente nas consultas. Non é necesario que o resultado dunha expresión estea almacenado previamente nunha columna.

Podemos utilizar os operadores aritméticos habituais:

Operador	Operación
`+`	Suma.
`-`	Resta.
`*`	Multiplicación.
`/`	División.
`%`	Resto dunha división.

Exemplo 1. Calcular o dobre das horas dun curso

```
SELECT nome, horas, horas * 2 AS DobreHoras
FROM cursos;
```

A consulta mostra o nome do curso, as horas almacenadas e unha nova columna calculada co dobre desas horas.

Exemplo 2. Calcular un incremento do 10 %

```
SELECT nome,
       horas,
       horas * 1.10 AS HorasIncrementadas
FROM cursos;
```

Exemplo 3. Realizar unha operación con varias columnas

```
SELECT nome,
       horas,
       horas / 10 AS ModulosEquivalentes
FROM cursos;
```

Os resultados das expresións poden recibir un alias para identificar facilmente o seu significado.

7.7. Combinación de LIMIT, alias e expresións

Todos estes elementos poden utilizarse nunha mesma consulta.

Exemplo 4. Mostrar os tres cursos con máis horas e calcular o seu dobre

```
SELECT nome AS Curso,
       horas AS Horas,
       horas * 2 AS DobreHoras
FROM cursos
ORDER BY horas DESC
LIMIT 3;
```

8. Funcións de resumo ou agregación

8.1. Que son as funcións de resumo?

As funcións de resumo, tamén chamadas funcións de agregación, permiten realizar cálculos sobre un conxunto de rexistros e devolver un único resultado.

A diferenza das consultas anteriores, que devolvían unha fila por cada rexistro seleccionado, estas funcións permiten obter información global dunha táboa.

As principais funcións de agregación son:

Función	Significado	Resultado
`COUNT()`	Conta os rexistros ou valores.	Número de elementos.
`SUM()`	Calcula a suma dos valores.	Suma total.
`AVG()`	Calcula a media aritmética.	Valor medio.
`MIN()`	Obtén o valor mínimo.	Valor máis pequeno.
`MAX()`	Obtén o valor máximo.	Valor máis grande.

8.2. COUNT(): contar rexistros

A función `COUNT()` permite contar o número de rexistros ou valores dunha consulta.

Podemos utilizar dúas formas habituais:

```
SELECT COUNT(*)
FROM nome_taboa;
```

Conta todas as filas devoltas pola consulta, independentemente de que algunha das súas columnas conteña valores NULL.

```
SELECT COUNT(columna)
FROM nome_taboa;
```

Conta unicamente os valores non nulos da columna indicada.

Exemplo 1. Contar o número total de alumnos

```
SELECT COUNT(*) AS TotalAlumnos
FROM alumnos;
```

Exemplo 2. Contar os alumnos que teñen un curso asignado

```
SELECT COUNT(curso_id) AS AlumnosConCurso
FROM alumnos;
```

Os rexistros nos que `curso_id` ten valor NULL non se contabilizan.

8.3. SUM(): sumar valores

A función `SUM()` calcula a suma dos valores dunha columna numérica.

```
SELECT SUM(columna)
FROM nome_taboa;
```

Exemplo 1. Calcular o total de horas de todos os cursos

```
SELECT SUM(horas) AS TotalHoras
FROM cursos;
```

8.4. AVG(): calcular a media

A función `AVG()` calcula a media aritmética dos valores dunha columna numérica.

```
SELECT AVG(columna)
FROM nome_taboa;
```

Exemplo 1. Calcular a idade media do alumnado

```
SELECT AVG(idade) AS IdadeMedia
FROM alumnos;
```

Os valores NULL non se teñen en conta no cálculo da media.

8.5. MIN(): obter o valor mínimo

A función `MIN()` devolve o valor máis pequeno dunha columna.

Exemplo. Consultar a menor idade do alumnado

```
SELECT MIN(idade) AS IdadeMinima
FROM alumnos;
```

8.6. MAX(): obter o valor máximo

A función `MAX()` devolve o valor máis grande dunha columna.

Exemplo. Consultar o maior número de horas dun curso

```
SELECT MAX(horas) AS MaximoHoras
FROM cursos;
```

8.7. Características das funcións de agregación

É importante ter en conta algunhas características do seu funcionamento:

- As funcións de agregación traballan sobre un conxunto de rexistros.
- Devolven normalmente un único resultado cando non existe agrupamento.
- Os valores NULL non se teñen en conta nos cálculos de `SUM()`, `AVG()`, `MIN()` e `MAX()`.
- `COUNT(*)` conta todas as filas, incluídas aquelas que conteñen valores NULL.
- `COUNT(columna)` só conta os valores non nulos.

> **Atención:** As funcións `SUM()` e `AVG()` deben utilizarse con columnas numéricas. Non teñen sentido para calcular operacións aritméticas sobre nomes ou descricións.

9. Agrupamento de rexistros con GROUP BY

9.1. Que é o agrupamento?

As funcións de agregación permiten obter resultados globais dunha táboa. Non obstante, en moitas situacións necesitamos realizar eses cálculos por grupos.

A cláusula `GROUP BY` permite agrupar os rexistros que teñen o mesmo valor nunha ou varias columnas.

Unha vez creados os grupos, podemos aplicar funcións de agregación a cada un deles.

```
SELECT columna, funcion_agregacion()
FROM nome_taboa
GROUP BY columna;
```

9.2. Agrupar por unha columna

Supoñamos que queremos saber cantos alumnos pertencen a cada curso. Para iso debemos agrupar os rexistros polo identificador do curso e contar os alumnos de cada grupo.

Exemplo 1. Número de alumnos por curso

```
SELECT curso_id, COUNT(*) AS TotalAlumnos
FROM alumnos
GROUP BY curso_id;
```

A consulta realiza as seguintes operacións:

1. Agrupa os alumnos segundo o valor de `curso_id`.
2. Conta os rexistros que pertencen a cada grupo.
3. Devolve unha fila por cada grupo.

9.3. Agrupar utilizando outras funcións de agregación

Podemos combinar `GROUP BY` con calquera das funcións de resumo estudadas anteriormente.

Exemplo 2. Idade media dos alumnos por curso

```
SELECT curso_id,
       AVG(idade) AS IdadeMedia
FROM alumnos
GROUP BY curso_id;
```

Exemplo 3. Idade mínima e máxima por curso

```
SELECT curso_id,
       MIN(idade) AS IdadeMinima,
       MAX(idade) AS IdadeMaxima
FROM alumnos
GROUP BY curso_id;
```

Exemplo 4. Número de alumnos e idade media por curso

```
SELECT curso_id,
       COUNT(*) AS TotalAlumnos,
       AVG(idade) AS IdadeMedia
FROM alumnos
GROUP BY curso_id;
```

9.4. Agrupamento por varias columnas

Podemos agrupar os rexistros utilizando máis dunha columna. Neste caso, os grupos fórmanse tendo en conta a combinación dos valores de todas as columnas indicadas.

```
SELECT columna1, columna2, COUNT(*)
FROM nome_taboa
GROUP BY columna1, columna2;
```

Exemplo. Agrupar os alumnos por curso e idade

```
SELECT curso_id,
       idade,
       COUNT(*) AS TotalAlumnos
FROM alumnos
GROUP BY curso_id, idade;
```

A consulta crea un grupo para cada combinación diferente de curso e idade.

9.5. Agrupamento e valores NULL

Cando agrupamos por unha columna que contén valores NULL, todos os rexistros que teñen NULL nesa columna forman parte dun mesmo grupo.

Exemplo. Contar alumnos por curso, incluídos os que non teñen curso asignado

```
SELECT curso_id,
       COUNT(*) AS TotalAlumnos
FROM alumnos
GROUP BY curso_id;
```

Se existen alumnos sen curso asignado, aparecerá un grupo no que `curso_id` será NULL.

9.6. Regras de uso de GROUP BY

Cando utilizamos `GROUP BY`, debemos ter coidado coas columnas que aparecen no `SELECT`.

Como regra xeral, as columnas seleccionadas deben:

- Estar incluídas na cláusula `GROUP BY`.
- Ou formar parte dunha función de agregación.

Consulta correcta

```
SELECT curso_id, COUNT(*) AS TotalAlumnos
FROM alumnos
GROUP BY curso_id;
```

Consulta incorrecta ou non recomendable:

```
SELECT curso_id, nome, COUNT(*)
FROM alumnos
GROUP BY curso_id;
```

O nome pode ter varios valores dentro dun mesmo grupo. Polo tanto, non existe un único valor de `nome` que represente ese grupo.

MariaDB/MySQL pode permitir este tipo de consultas dependendo da configuración de `sql_mode`, pero o resultado pode ser indeterminado. Debemos evitar esta práctica.

10. Filtrado de grupos con HAVING

10.1. Que é HAVING?

A cláusula `HAVING` permite establecer condicións sobre os grupos creados mediante `GROUP BY`.

A diferenza de `WHERE`, que filtra rexistros individuais antes de realizar o agrupamento, `HAVING` filtra os grupos despois de realizar os cálculos de agregación.

```
SELECT columna, funcion_agregacion()
FROM nome_taboa
GROUP BY columna
HAVING condicion;
```

10.2. Exemplo de uso de HAVING

Exemplo 1. Cursos con máis de dous alumnos

```
SELECT curso_id,
       COUNT(*) AS TotalAlumnos
FROM alumnos
GROUP BY curso_id
HAVING COUNT(*) > 2;
```

Primeiro agrúpanse os alumnos por curso e cóntanse os rexistros de cada grupo. Despois só se mostran os grupos que teñen máis de dous alumnos.

10.3. WHERE fronte a HAVING

A principal diferenza entre estas dúas cláusulas é o momento no que se aplican e o tipo de información que permiten filtrar.

WHERE	HAVING
Filtra rexistros individuais.	Filtra grupos de rexistros.
Execútase antes do agrupamento.	Execútase despois do agrupamento.
Non se utiliza para filtrar directamente resultados de funcións de agregación.	Permite filtrar utilizando funcións de agregación.
Utilízase normalmente xunto con columnas individuais.	Utilízase habitualmente xunto con GROUP BY.

10.4. Exemplo comparativo de WHERE e HAVING

Supoñamos que queremos contar os alumnos maiores de idade que hai en cada curso.

Exemplo 1. Filtrar os alumnos antes de agrupar

```
SELECT curso_id,
       COUNT(*) AS TotalAlumnos
FROM alumnos
WHERE idade >= 18
GROUP BY curso_id;
```

Primeiro elimínanse os alumnos menores de idade. Despois agrúpanse os restantes por curso e cóntanse.

Agora queremos mostrar unicamente os cursos que teñen máis de dous alumnos maiores de idade.

Exemplo 2. Filtrar os grupos despois de agrupar

```
SELECT curso_id,
       COUNT(*) AS TotalAlumnos
FROM alumnos
WHERE idade >= 18
GROUP BY curso_id
HAVING COUNT(*) > 2;
```

Nesta consulta:

1. `WHERE` elimina os alumnos menores de idade.
2. `GROUP BY` agrupa os restantes por curso.
3. `COUNT()` conta os alumnos de cada curso.
4. `HAVING` conserva os grupos con máis de dous alumnos.

10.5. Utilización de varias condicións en HAVING

Podemos combinar varias condicións utilizando operadores lóxicos.

Exemplo. Cursos con máis de dous alumnos e idade media superior a 18

```
SELECT curso_id,
       COUNT(*) AS TotalAlumnos,
       AVG(idade) AS IdadeMedia
FROM alumnos
GROUP BY curso_id
HAVING COUNT(*) > 2
AND AVG(idade) > 18;
```

10.6. Estrutura completa dunha consulta con agrupamento

Cando combinamos as diferentes cláusulas nunha consulta, debemos respectar a seguinte orde:

```
SELECT columnas, funciones_agregacion
FROM nome_taboa
WHERE condicion_registros
GROUP BY columnas
HAVING condicion_grupos
ORDER BY columnas
LIMIT numero;
```

> **Para lembrar:**
>
> - `WHERE` : filtra os rexistros antes de agrupar.
> - `GROUP BY` : crea os grupos.
> - `HAVING` : filtra os grupos creados.
> - `ORDER BY` : ordena os resultados.
> - `LIMIT` : limita o número de resultados.

11. Consultas de resumo combinadas

11.1. Utilización de varias funcións de agregación

Nunha mesma consulta podemos utilizar varias funcións de agregación para obter diferentes indicadores sobre os datos.

Exemplo 1. Estatísticas xerais do alumnado

```
SELECT COUNT(*) AS TotalAlumnos,
       AVG(idade) AS IdadeMedia,
       MIN(idade) AS IdadeMinima,
       MAX(idade) AS IdadeMaxima
FROM alumnos;
```

A consulta devolve unha única fila con catro valores:

- Número total de alumnos.
- Idade media.
- Idade mínima.
- Idade máxima.

Exemplo 2. Estatísticas de horas dos cursos

```
SELECT COUNT(*) AS TotalCursos,
       SUM(horas) AS TotalHoras,
       AVG(horas) AS MediaHoras,
       MIN(horas) AS MinimoHoras,
       MAX(horas) AS MaximoHoras
FROM cursos;
```

11.2. Varias funcións de agregación con GROUP BY

Tamén podemos combinar varias funcións de agregación con `GROUP BY` para obter información detallada de cada grupo.

Exemplo 3. Estatísticas do alumnado por curso

```
SELECT curso_id,
       COUNT(*) AS TotalAlumnos,
       AVG(idade) AS IdadeMedia,
       MIN(idade) AS IdadeMinima,
       MAX(idade) AS IdadeMaxima
FROM alumnos
GROUP BY curso_id;
```

11.3. Consultas de resumo con WHERE, GROUP BY e HAVING

Podemos combinar as cláusulas estudadas para resolver consultas que requiren varios pasos de filtrado e agrupamento.

Exemplo 4. Estatísticas dos alumnos maiores de idade por curso

```
SELECT curso_id,
       COUNT(*) AS TotalAlumnos,
       AVG(idade) AS IdadeMedia
FROM alumnos
WHERE idade >= 18
GROUP BY curso_id
HAVING COUNT(*) >= 2
ORDER BY TotalAlumnos DESC;
```

A consulta:

1. Selecciona os alumnos maiores de idade.

2. Agrúpaos polo identificador do curso.
3. Conta os alumnos e calcula a idade media.
4. Conserva os cursos con polo menos dous alumnos.
5. Ordena os resultados polo número de alumnos.

11.4. Consultas de resumo con expresións

As expresións tamén poden utilizarse para presentar resultados derivados das funcións de agregación.

Exemplo 5. Calcular a diferenza entre a idade máxima e mínima de cada curso

```
SELECT curso_id,
       MAX(idade) AS IdadeMaxima,
       MIN(idade) AS IdadeMinima,
       MAX(idade) - MIN(idade) AS DiferenzaIdades
FROM alumnos
GROUP BY curso_id;
```

11.5. Consideracións importantes

- As funcións de agregación permiten resumir grandes cantidades de información.
- `GROUP BY` permite realizar cálculos independentes para cada grupo.
- `WHERE` filtra os rexistros antes de agrupar.
- `HAVING` filtra os grupos despois de realizar as operacións de agregación.
- Os alias facilitan a interpretación dos resultados.

11.6. Exercicios de consolidación: funcións de resumo

1. Obter o número total de alumnos.
2. Calcular a idade media do alumnado.
3. Obter a idade mínima e máxima dos alumnos.
4. Calcular o total de horas de todos os cursos.
5. Calcular a media de horas dos cursos.
6. Contar cantos alumnos teñen un curso asignado.
7. Contar o número de alumnos que hai en cada curso.
8. Calcular a idade media dos alumnos de cada curso.
9. Obter a idade mínima e máxima de cada curso.
10. Mostrar os cursos que teñen máis de dous alumnos.

11. Mostrar os cursos nos que a idade media é superior a 18 anos.
12. Contar os alumnos maiores de idade de cada curso.
13. Mostrar os cursos que teñen polo menos tres alumnos maiores de idade.
14. Ordenar os cursos polo número de alumnos de maior a menor.

12. Relacións entre táboas e composicións internas

12.1. Por que necesitamos consultar varias táboas?

Nunha base de datos relacional, a información adoita estar distribuída en varias táboas relacionadas entre si.

Esta organización permite evitar a repetición innecesaria de información e manter a coherencia dos datos.

Por exemplo, nunha base de datos dun centro educativo podemos ter unha táboa para almacenar os alumnos e outra para almacenar os cursos. Cada alumno pertence a un curso, polo que ambas as táboas están relacionadas.

12.2. Claves primarias

Unha clave primaria é unha columna ou un conxunto de columnas que permite identificar de maneira única cada rexistro dunha táboa.

As principais características dunha clave primaria son:

- Os seus valores deben ser únicos.
- Non pode conter valores NULL.
- Cada táboa ten unha única clave primaria, que pode estar formada por unha ou varias columnas.
- Permite identificar cada rexistro de maneira inequívoca.

Exemplo. Táboa cursos

```
CREATE TABLE cursos (
    id INT AUTO_INCREMENT,
    nome VARCHAR(50) NOT NULL,
    horas INT,
    PRIMARY KEY (id)
);
```

A columna `id` é a clave primaria da táboa. Cada curso terá un identificador diferente.

12.3. Claves foráneas

Unha clave foránea é unha columna ou un conxunto de columnas que establece unha relación entre dúas táboas.

A clave foránea referencia unha clave primaria ou unha clave única doutra táboa. Deste xeito podemos relacionar os rexistros dunha táboa cos rexistros doutra.

Exemplo. Táboa alumnos

```
CREATE TABLE alumnos (
    id INT AUTO_INCREMENT,
    nome VARCHAR(100) NOT NULL,
    idade INT,
    curso_id INT,
    PRIMARY KEY (id),
    FOREIGN KEY (curso_id)
        REFERENCES cursos(id)
);
```

A columna `curso_id` é unha clave foránea que referencia a columna `id` da táboa `cursos`.

12.4. Representación da relación

A relación entre as táboas `cursos` e `alumnos` é de tipo un a moitos (1:N):

- Un curso pode ter varios alumnos.
- Cada alumno pertence como máximo a un curso.
- A columna `cursos.id` é a clave primaria.
- A columna `alumnos.curso_id` é a clave foránea.

12.5. Inserción de datos relacionados

Para comprender como se relacionan as táboas, podemos introducir algúns rexistros de exemplo.

```
INSERT INTO cursos (nome, horas)
VALUES
('DAW1', 100),
('DAW2', 100),
('DAM1', 200),
('DAM2', 200);
```

A continuación, inserimos alumnos indicando o identificador do curso ao que pertencen.

```
INSERT INTO alumnos (nome, idade, curso_id)
VALUES
('Ana García', 20, 1),
('Luis López', 19, 1),
('Marta Fernández', 21, 2),
('Pedro Rodríguez', 18, 2),
('Laura Martínez', 22, 3);
```

O valor de `curso_id` debe corresponder cun identificador existente na táboa `cursos`. Deste xeito mantense a integridade referencial.

12.6. Consultas sobre varias táboas

Se realizamos unha consulta sobre a táboa `alumnos`, podemos recuperar o identificador do curso ao que pertence cada alumno.

```
SELECT *
FROM alumnos;
```

Non obstante, o identificador non nos permite coñecer directamente o nome do curso. Para obter esa información necesitamos consultar tamén a táboa `cursos`.

As composicións permiten combinar información de varias táboas nunha mesma consulta.

12.7. Composición interna: INNER JOIN

A composición interna, ou `INNER JOIN`, permite combinar rexistros de dúas ou máis táboas cando existe unha correspondencia entre os valores das columnas indicadas na condición de unión.

Só se devolven os rexistros que teñen correspondencia nas táboas relacionadas.

```
SELECT columnas
FROM taboa1
INNER JOIN taboa2
ON taboa1.columna = taboa2.columna;
```

A cláusula `ON` establece a condición que permite relacionar os rexistros das dúas táboas.

12.8. Primeiro exemplo de INNER JOIN

Exemplo 1. Mostrar os alumnos co nome do curso ao que pertencen

```
SELECT alumnos.nome,
       cursos.nome
FROM alumnos
INNER JOIN cursos
ON alumnos.curso_id = cursos.id;
```

A consulta combina a información das dúas táboas:

- Recupera o nome do alumno desde `alumnos`.
- Recupera o nome do curso desde `cursos`.
- Relaciona ambas as táboas mediante as columnas `curso_id` e `id`.

12.9. Uso de alias nas composicións

Cando traballamos con varias táboas é recomendable utilizar alias para simplificar as consultas.

```
SELECT a.nome,
       c.nome
FROM alumnos AS a
INNER JOIN cursos AS c
ON a.curso_id = c.id;
```

Neste exemplo:

- `a` representa a táboa `alumnos`.
- `c` representa a táboa `cursos`.
- `a.curso_id` identifica a clave foránea.
- `c.id` identifica a clave primaria.

Os alias tamén permiten diferenciar columnas que teñen o mesmo nome en diferentes táboas.

Exemplo 2. Consultar os datos do alumno e do curso con alias de columnas

```
SELECT a.nome AS Alumno,
       a.idade AS Idade,
       c.nome AS Curso,
       c.horas AS Horas
FROM alumnos AS a
INNER JOIN cursos AS c
ON a.curso_id = c.id;
```

12.10. INNER JOIN con condicións de filtrado

Podemos combinar as composicións internas coa cláusula `WHERE` para filtrar os resultados.

Exemplo 3. Mostrar os alumnos do curso DAW1

```
SELECT a.nome AS Alumno,
       c.nome AS Curso
FROM alumnos AS a
INNER JOIN cursos AS c
ON a.curso_id = c.id
WHERE c.nome = 'DAW1';
```

Exemplo 4. Mostrar os alumnos maiores de 18 anos e o seu curso

```
SELECT a.nome AS Alumno,
       a.idade AS Idade,
       c.nome AS Curso
FROM alumnos AS a
INNER JOIN cursos AS c
ON a.curso_id = c.id
WHERE a.idade > 18;
```

12.11. INNER JOIN con ordenación

Tamén podemos utilizar `ORDER BY` para ordenar os resultados dunha composición interna.

Exemplo 5. Mostrar os alumnos ordenados polo nome do curso

```
SELECT a.nome AS Alumno,
       c.nome AS Curso
FROM alumnos AS a
INNER JOIN cursos AS c
ON a.curso_id = c.id
ORDER BY c.nome, a.nome;
```

12.12. INNER JOIN con funcións de resumo

Podemos combinar as composicións internas coas funcións de agregación para obter información resumida de varias táboas.

Exemplo 6. Contar o número de alumnos de cada curso

```
SELECT c.nome AS Curso,
       COUNT(a.id) AS TotalAlumnos
FROM cursos AS c
INNER JOIN alumnos AS a
ON c.id = a.curso_id
GROUP BY c.id, c.nome;
```

A consulta relaciona os alumnos cos seus cursos e agrupa os resultados polo identificador e nome do curso.

12.13. Características de INNER JOIN

- Permite recuperar información almacenada en varias táboas.
- Necesita unha condición de unión mediante `ON`.
- Normalmente utiliza relacións entre claves primarias e foráneas.
- Só devolve os rexistros que teñen correspondencia nas táboas unidas.
- Permite combinarse con `WHERE`, `GROUP BY`, `HAVING` e `ORDER BY`.

> **Importante:** Se existen alumnos sen curso asignado, ou cun identificador que non ten correspondencia na táboa de cursos, eses alumnos non aparecerán no resultado dun `INNER JOIN`. Isto débese a que a composición interna só devolve coincidencias.

12.14. Exercicios de consolidación: INNER JOIN

1. Mostrar o nome de cada alumno xunto co identificador do curso ao que pertence.
2. Mostrar o nome de cada alumno e o nome do seu curso.
3. Mostrar o nome do alumno, a súa idade e o nome do curso.
4. Mostrar os alumnos que pertencen ao curso DAW1.
5. Mostrar os alumnos maiores de 18 anos xunto co nome do seu curso.
6. Mostrar todos os alumnos ordenados polo nome do curso.
7. Mostrar os alumnos do curso DAM1 ordenados alfabeticamente.
8. Contar cantos alumnos ten cada curso.
9. Calcular a idade media dos alumnos de cada curso.
10. Mostrar os cursos que teñen máis de dous alumnos utilizando GROUP BY e HAVING.
11. Mostrar o nome do curso, o número de alumnos e a idade media de cada curso.

13. Resumo das consultas estudadas

Elemento	Función	Exemplo
`LIMIT`	Limitar o número de resultados.	`LIMIT 5`
`OFFSET`	Saltar un número de rexistros.	`LIMIT 5 OFFSET 10`
`AS`	Asignar un alias.	`nome AS Alumno`
`COUNT()`	Contar rexistros ou valores.	`COUNT(*)`
`SUM()`	Sumar valores.	`SUM(horas)`
`AVG()`	Calcular unha media.	`AVG(idade)`
`MIN()`	Obter o mínimo.	`MIN(idade)`
`MAX()`	Obter o máximo.	`MAX(idade)`
`GROUP BY`	Agrupar rexistros.	`GROUP BY curso_id`
`HAVING`	Filtrar grupos.	`HAVING COUNT(*) > 2`
`INNER JOIN`	Combinar rexistros relacionados.	`INNER JOIN cursos`
`ON`	Establecer a condición de unión.	`ON a.curso_id = c.id`

13.1. Estrutura completa dunha consulta

As consultas SQL poden combinar todas as cláusulas estudadas ata o momento. A estrutura xeral é:

```
SELECT columnas, funciones_agregacion
FROM taboa1
INNER JOIN taboa2
    ON condicion_union
WHERE condicion_registros
GROUP BY columnas
HAVING condicion_grupos
ORDER BY columnas
LIMIT numero;
```

Exemplo final. Resumo do alumnado por curso

Queremos obter o nome de cada curso, o número de alumnos maiores de idade, a súa idade media e mostrar unicamente os cursos que teñan polo menos dous alumnos. Os resultados deben ordenarse polo número de alumnos de maior a menor.

```
SELECT c.nome AS Curso,
       COUNT(a.id) AS TotalAlumnos,
       AVG(a.idade) AS IdadeMedia
FROM cursos AS c
INNER JOIN alumnos AS a
    ON c.id = a.curso_id
WHERE a.idade >= 18
GROUP BY c.id, c.nome
HAVING COUNT(a.id) >= 2
ORDER BY TotalAlumnos DESC;
```

Esta consulta combina:

- Selección de columnas e alias.
- Composición interna de dúas táboas.
- Filtrado de rexistros mediante WHERE.
- Funcións de agregación.
- Agrupamento con GROUP BY.
- Filtrado de grupos con HAVING.
- Ordenación con ORDER BY.